

4.3 Deploy your frontend with AWS Amplify

📌 this section we are deploying my project onto AWS amplify. This is because like the ticket says 'Code that only runs on your laptop isn't finished. Getting comfortable with the deploy step now means it isn't a surprise later.'

It says this because the code is only deployed locally on my laptop so by deploying it on AWS amplify the project now becomes deployed on the cloud and it becomes publicly available.

Learning objective

What does Amplify actually do for you?

AWS Amplify gives me a simple way to **build, host and deploy my frontend application** without manually setting up web servers. It connects to my Git repository, runs the application's build process and publishes the finished website to a public URL. It also gives me continuous deployment, meaning future versions of my frontend can be deployed automatically.

How does it build from a git branch, and what happens when you push?

Amplify is connected to a specific Git branch, which in my project is the **main branch**. It downloads the code from that branch, installs the required packages, runs `npm run build` and then deploys the completed application. When I push a new commit to `main`, Amplify automatically detects the change and starts a new build and deployment, meaning I do not have to manually redeploy the website every time I update it.

Where do environment variables go, and why must the API URL never be hardcoded in your source?

Environment variables such as `API_BASE_URL` are stored in the **Amplify environment variable configuration** rather than directly inside my source code. Locally, I store them in `.env.local`, which is ignored by Git. Keeping the API URL in configuration means I can change which backend the frontend connects to without changing the application code, and it prevents deployment-specific values from being unnecessarily hardcoded throughout the project.

What's the difference between a build-time and a run-time environment variable in Next.js? This one catches people out.

A **build-time environment variable** is available while `npm run build` is creating the Next.js application and can become part of the generated application output. A **run-time environment**

variable is read later while the deployed server-side application is actually handling requests. This distinction was important in my project because the Next.js API proxy runs on the server and needs access to the backend configuration when the deployed application is running.

Task



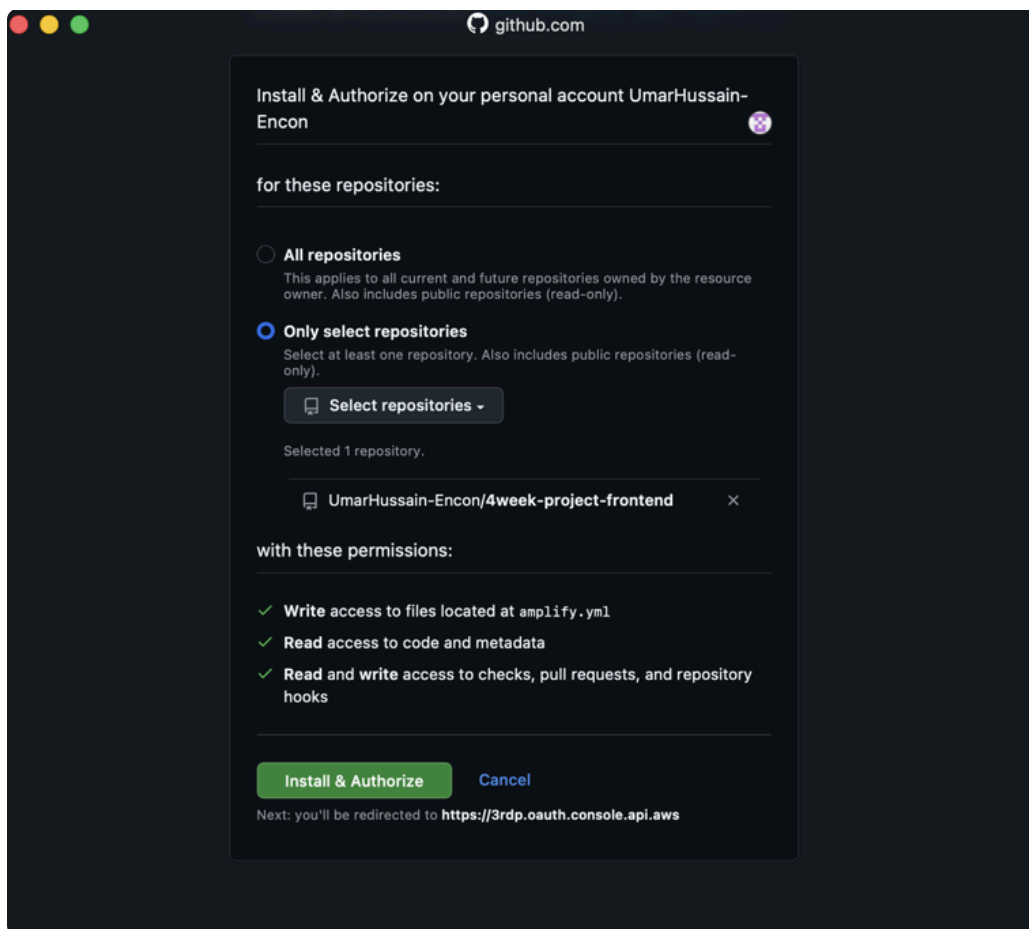
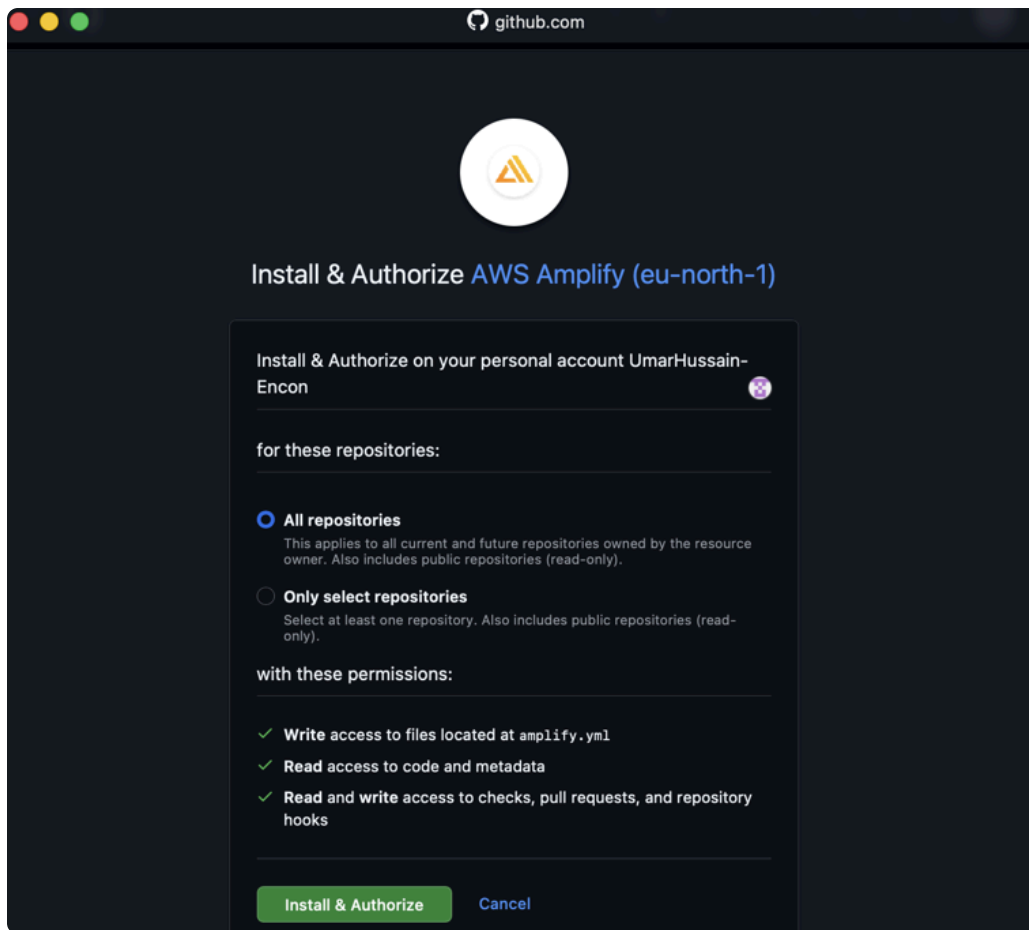
- Connect your repo to Amplify
- Get the build passing
- Deploy it
- Confirm the deployed site talks to your real API, not to localhost

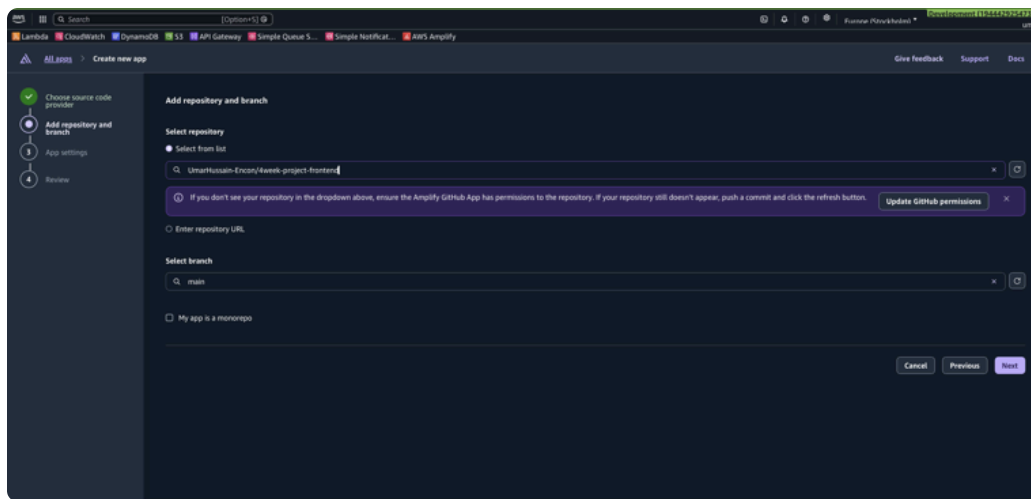
The first thing I did was down grade the Next version from 16 to 15 this is because AWS amplify is not compatible with Next.js 15. Therefore I ran:

```
npm install next@15 -then ran→ npm list next react react-dom -then ran→ npm run build
```

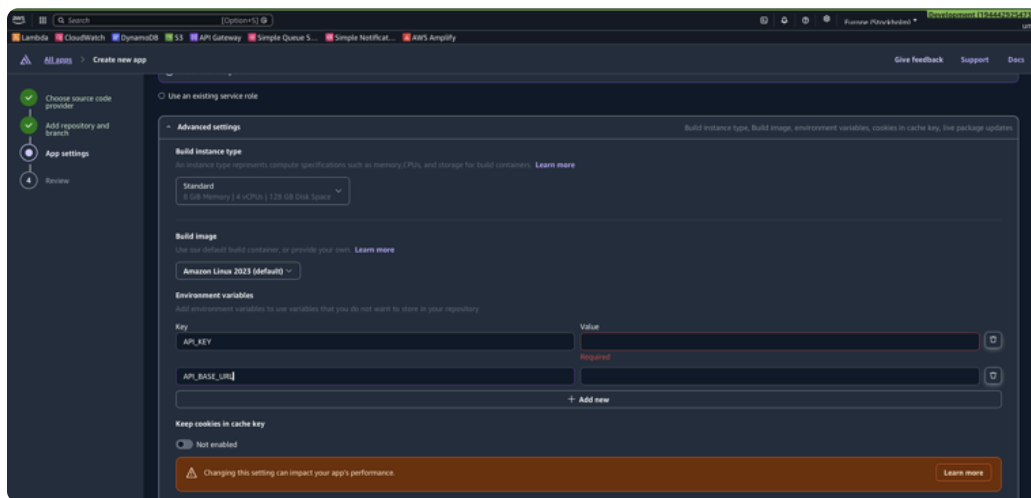
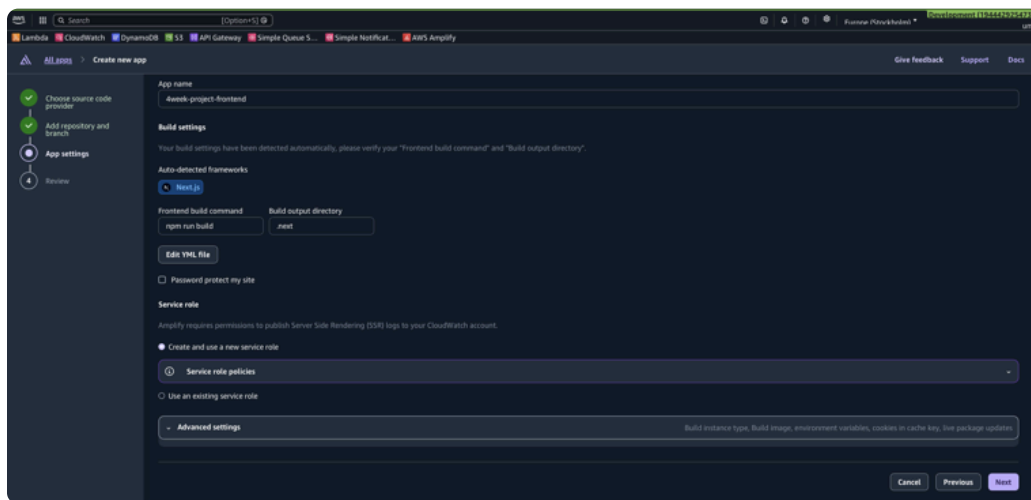
This didn't affect anything, the code didn't break, the application still work and so did the backend. Now we can deploy the application on AWS amplify.

I first connected my 4week-project-frontend, I changed the allow any repo to just the specific 4week-project-front end one in my Git account.

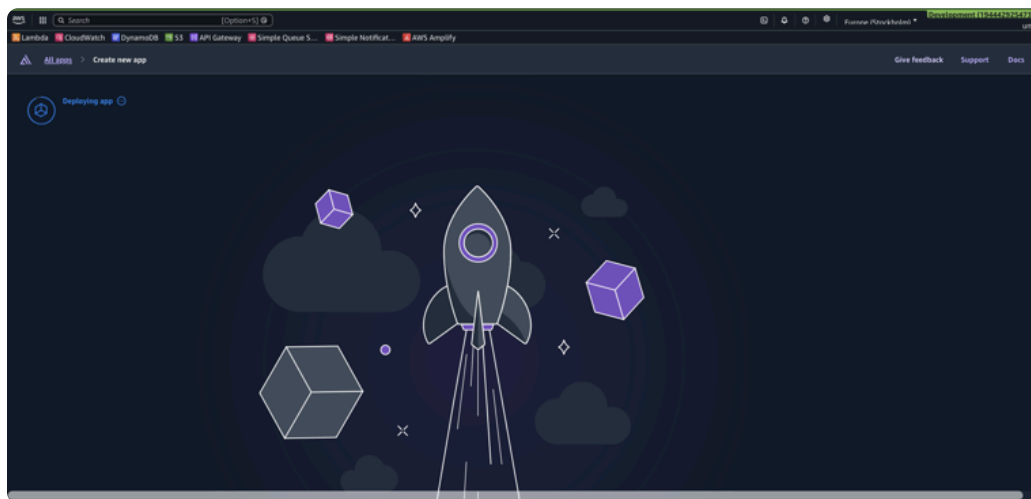
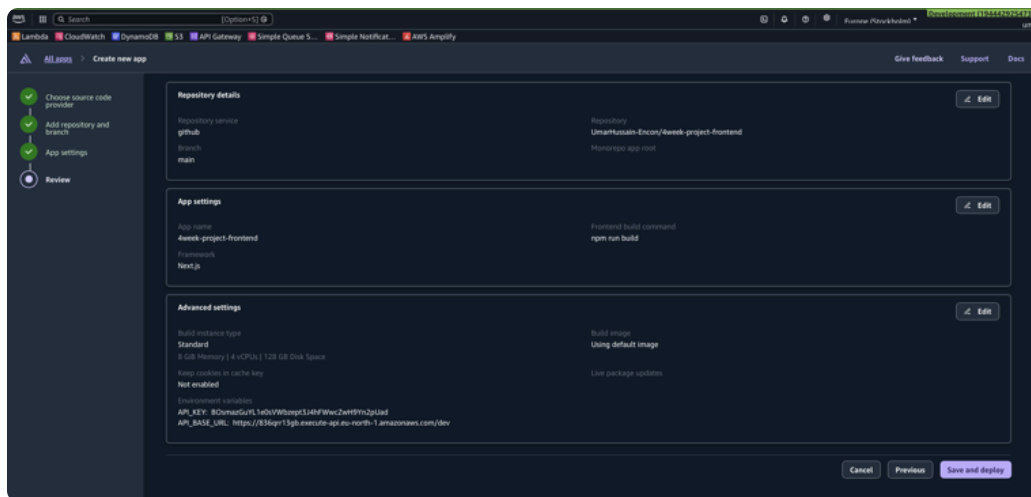




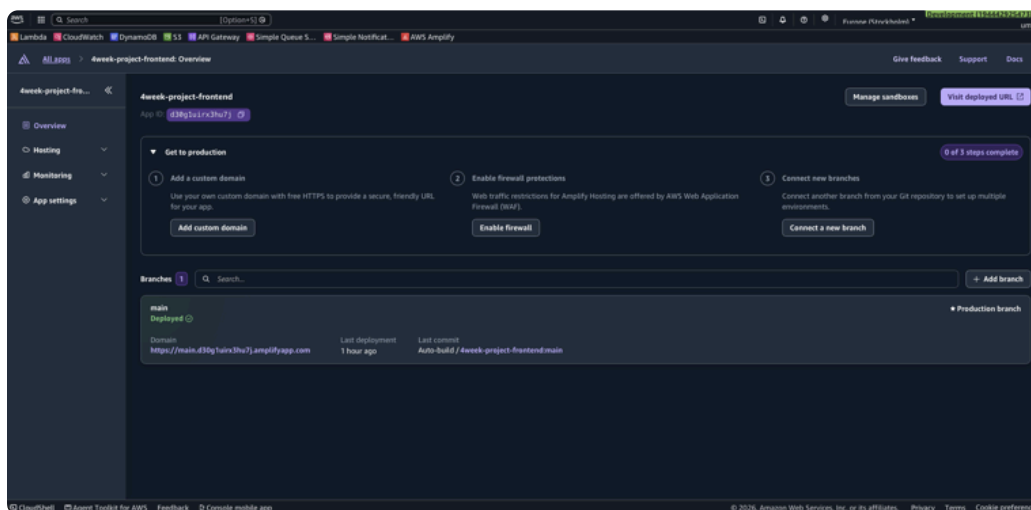
Then I went onto the configuration page and edited the YAML file; and I also inputted the API key and API URL.



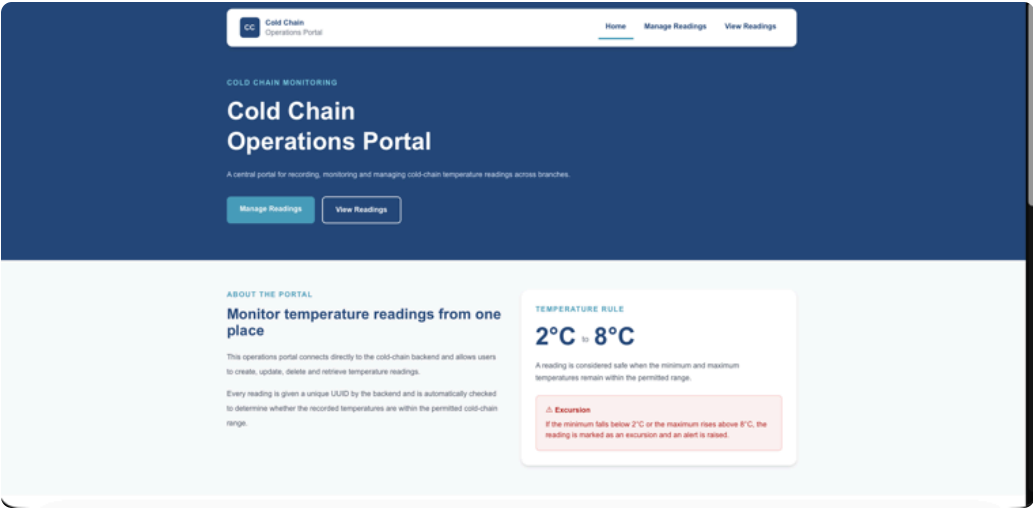
Then after looking over the review page to make sure everything was working correctly, I deployed the Amplify.



Then I copied the URL it gave me and the application was running on a public URL and not just off my local device.



<https://main.d30g1uirx3hu7j.amplifyapp.com>



THE PROJECT IS NOW COMPLETED